A Comparative Analysis of Q-Learning, SARSA, and Value Iteration in a Navigation Task

Jose Luna jlunabo1@jh.edu
Applied and Computational Mathematics
Johns Hopkins University

Abstract

This study evaluates the performance of Q-learning, SARSA, and Value Iteration on a set of racetrack navigation tasks with different layouts and crash-handling modes. Each algorithm was tuned and assessed over multiple trials, with performance measured by steps-to-finish, variance, and convergence speed. Results show that Value Iteration consistently achieved the lowest average steps-to-finish and exhibited minimal variance across trials, confirming its advantage in deterministic, fully known environments. Q-learning outperformed SARSA in mean efficiency on most tracks and converged faster in training, while SARSA failed to show a variance advantage. These findings suggest that, under identical exploration schedules, Q-learning's off-policy nature does not inherently increase instability, and model-based planning remains superior when the environment model is available.

Keywords: Reinforcement Learning, Q-Learning, SARSA, Value Iteration

1 Introduction

Over the past decade, reinforcement learning (RL) has achieved remarkable successes across a wide range of domains, including robotics, game playing, and even autonomous driving. These achievements underscore the versatility of RL in environments that are dynamic, stochastic, and high-dimensional. The purpose of this project is to explore RL in the context of a *racetrack problem*, where an agent must navigate from a starting line to a finish line in as few time steps as possible while avoiding collisions with track boundaries. The agent's state includes both position and velocity, and its actions are discrete accelerations in the x and y directions. Moreover, the environment introduces randomness: any attempted acceleration has a 20% chance of failing, forcing the agent to account for uncertainty in planning. We implement and compare three algorithms for training an RL agent:

- **Value Iteration** (VI): A model-based dynamic programming method that uses the full environment model to compute an optimal policy.
- **Q-Learning**: A model-free, *off-policy* temporal-difference method that learns the optimal *state-action value function* $Q(s, a)$, which estimates the expected cumulative reward from taking action a in state s and thereafter following the greedy policy with respect to Q.
- **SARSA**: A model-free, *on-policy* temporal-difference method that also learns a state-action value function $Q(s, a)$, but updates it using the actual actions taken by the current policy.

Two crash handling modes are tested:

- **Nearest Point**: Upon crashing, the car is moved to the nearest valid track square and its velocity reset to $(0, 0)$.
- **Restart**: Upon crashing, the car is returned to the closest start position and its velocity reset to $(0, 0)$ (we only test this on the R-track, shown below)

We conduct experiments on four different racetracks, shown below:

Figure 1: L-track

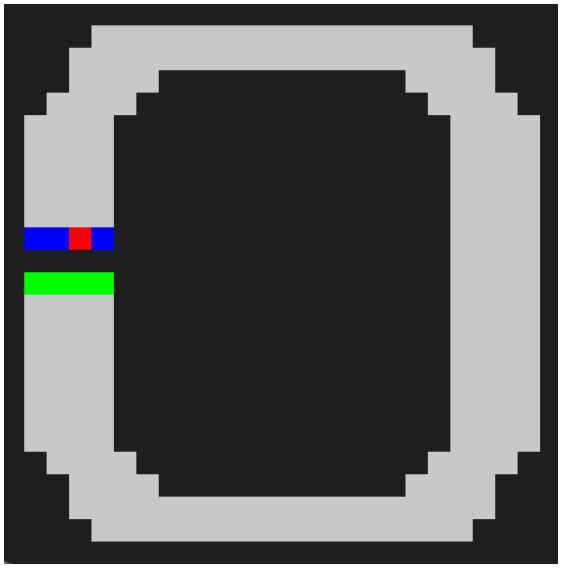

Figure 2: O-track

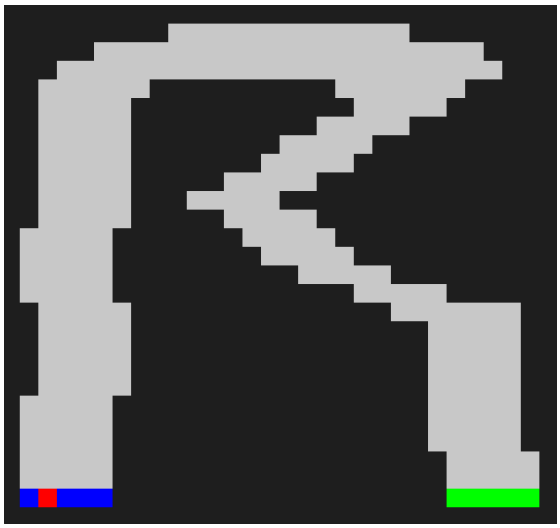

Figure 3: R-track

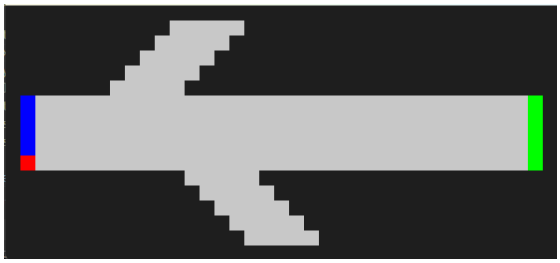

Figure 4: W-track

1.1 Hypotheses

To guide our exploration, we formulate two hypotheses to test, as well as the rationales behind them:

1. Across all tracks, Value Iteration will produce the lowest average steps to finish.
   - *Rationale:* VI computes the optimal policy exactly from the full model, avoiding the sampling noise and model-free methods.
2. SARSA will achieve lower *variance* in steps to finish than Q-Learning, while Q-Learning will achieve a lower *mean* cost after sufficient training.
   - *Rationale:* SARSA's on-policy updates favor safer, more consistent behavior, while Q-Learning's off-policy updates favor faster but riskier policies.

2 Algorithms

Q-Learning and SARSA aim to learn a *state-action value function* $Q(s, a)$, where s is current state (position and velocity of the car), a is an action taken (acceleration in x and y), and Q is the expected total cost (discounted over time by a factor $\gamma \in [0, 1)$) from taking action a in state s, then following a given policy until the race finishes. In our problem, the cost for each step is 1, and the finish line has cost 0.

For Value Iteration, we work with the *state value function* $V(s)$ instead of $Q(s, a)$. $V(s)$ is the expected total discounted cost starting from state s if we always act optimally.

2.1 Q-Learning

Q-Learning estimates the optimal $Q(s, a)$ without knowing the environment's transition probabilities. It is off-policy, meaning it updates $Q(s, a)$ toward the best action in the next state, regardless of which action is actually chosen during exploration. Mathematically, after taking action a in state s and landing in s' with cost c:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha\Big[c + \gamma \min_{a'} Q(s', a')\Big],$$

where α is the learning rate and γ is the discount factor.

Q-Learning Pseudocode

1. Initialize $Q(s, a)$ to 0 for all s, a.
2. For each episode:
   (a) Start from a random start state s.
   (b) Repeat until the finish line is reached:
      i. Choose action a with ϵ-greedy exploration:
         · With probability ϵ, select a random action (exploration).
         · With probability $1 - \epsilon$, select $\min_a Q(s, a)$ (exploitation).
      ii. Apply a, observe cost c and next state s'.
      iii. Update $Q(s, a)$ using the equation above.
      iv. Set $s \leftarrow s'$.

2.2 SARSA

SARSA also estimates $Q(s, a)$, but it is on-policy: it updates based on the action actually taken next. The update after taking a in s, then choosing a' in s' with cost c is:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha\Big[c + \gamma Q(s', a')\Big].$$

SARSA Pseudocode

1. Initialize $Q(s, a)$ to 0 for all s, a.
2. For each episode:
   (a) Start from a random start state s.
   (b) Choose a via ϵ-greedy exploration (as described above).
   (c) Repeat until the finish line is reached:
      i. Apply a, observe cost c and next state s'.
      ii. Choose a' for s' via ϵ-greedy exploration.
      iii. Update $Q(s, a)$ using the equation above.
      iv. Set $s \leftarrow s'$ and $a \leftarrow a'$.

2.3 Value Iteration

Value Iteration assumes knowledge of the transition probabilities $P(s'|s, a)$, which in this project are given. It computes the optimal state value $V(s)$ by repeatedly applying the Bellman optimality equation:

$$V_{t+1}(s) = \min_a \Big[c(s, a) + \gamma \sum_{s'} P(s'|s, a)V_t(s')\Big],$$

where $c(s, a)$ is the expected immediate cost for action a in state s.

The algorithm stops when the maximum absolute change in $V(s)$ across all states between two successive sweeps is less than a small threshold θ:

$$\max_s \big|V_{t+1}(s) - V_k(s)\big| < \theta.$$

This ensures that further updates would not significantly change the value estimates. Once $V(s)$ has converged, we extract the optimal policy:

$$\pi(s) = \arg\min_a \Big[c(s, a) + \gamma \sum_{s'} P(s'|s, a)V(s')\Big].$$

Value Iteration Pseudocode

1. Initialize $V(s)$ to 0 for all s.
2. Repeat until $\max_s |V_{t+1}(s) - V_t(s)| < \theta$:
   (a) For each state s, update $V(s)$ using the Bellman equation above.
3. For each state s, find the action a that minimizes $c(s, a) + \gamma \sum_{s'} P(s'|s, a)V(s')$, and set $\pi(s)$ to that action.

2.4 Exploration Schedules

For both Q-Learning and SARSA, we use an ϵ-greedy exploration strategy: with probability ϵ, we select a random action (known as *exploration*); otherwise, we select the action that minimizes the current estimate of the total cost (known as *exploitation*). We implement ϵ decay using a linear schedule:

$$\epsilon(t) = \begin{cases} e_0 - \dfrac{e_0 - e_{\min}}{T} \cdot t, & 0 \le t < T, \\ e_{\min}, & t \ge T, \end{cases}$$

where e_0 is the initial exploration probability, $e_{\min}$ is the minimum exploration probability (set to 0.05), T is the number of episodes over which to decay ϵ (set to $num_episodes/2$), and t is the current episode index.

3 Experimental Approach

3.1 Hyperparameter Tuning

We tune the key hyperparameters for Q-Learning, SARSA, and Value Iteration by performing a grid search over predefined parameter sets, running each configuration multiple times, and averaging performance.

Q-Learning and SARSA: For both algorithms, we tune:

- Learning rate $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$
- Discount factor $\gamma \in \{0.90, 0.95, 0.99\}$
- Initial exploration probability $e_0 \in \{0.2, 0.3, 0.4\}$

The tuning process consists of two stages: an initial coarse search over all parameter combination, and a focused search over promising regions in hyperparameter space based on initial results. Each configuration is trained for a fixed number of episodes: 20,000 episodes for the L-track and W-track, and 50,000 episodes for the larger O-track and R-track.

Value Iteration: Since Value Iteration is model-based and does not involve learning rates or exploration parameters, we tune only the discount factor:

$$\gamma \in \{0.90, 0.95, 0.99\}$$

Each run proceeds until the value function $V(s)$ converges within a threshold $\theta = 0.001$.

Evaluation procedure: For each hyperparameter combination, we run the algorithm three times with different random seeds and average the resulting mean episode cost (average steps to finish) to obtain the final score for that configuration. The best parameters for each track and crash mode are selected based on this averaged cost.

3.1.1 Optimal Hyperparameters Found

Algorithm	Track	Crash Mode	α	γ	ϵ_0
Q	L-track	nearest	0.550	0.990	0.200
	O-track	nearest	0.500	0.900	0.200
	W-track	nearest	0.600	0.990	0.250
	R-track	nearest	0.300	0.950	0.400
	R-track	restart	0.275	0.900	0.300
SARSA	L-track	nearest	0.500	0.900	0.400
	O-track	nearest	0.500	0.900	0.400
	W-track	nearest	0.500	0.900	0.300
	R-track	nearest	0.300	0.900	0.400
	R-track	restart	0.250	0.900	0.200
VI	L-track	nearest	—	0.500	—
	O-track	nearest	—	0.900	—
	W-track	nearest	—	0.500	—
	R-track	nearest	—	0.900	—
	R-track	restart	—	0.900	—

3.2 Experiments

3.2.1 Training Protocol

We evaluate the three algorithms on four racetracks (L, O, W, R). For the R-track, we test two crash modes (*nearest* and *restart*); for the other tracks, we use *nearest*. Each algorithm is trained with the optimal hyperparameters from Section 3.1.1, for the same number of training episodes as were used for hyperparameter tuning. To obtain reasonable statistics, we run 10 independent random seeds for each (algorithm, track, crash mode) configuration.

3.2.2 Evaluation Protocol

After training with a given seed, the resulting policy is evaluated over 100 greedy episodes ($\epsilon = 0$). The performance metric recorded is the number of *steps-to-finish* in each episode.

4 Results

Trials where the agent hit the maximum step limit (750 steps) were excluded from performance statistics, as these represent incomplete runs rather than valid completion times.

Algorithm	Track	Mode	Mean Steps	Std. Dev.	Success Rate
Q-Learning	L-track	nearest	12.751	2.376	0.999
	O-track	nearest	29.614	3.025	0.999
	W-track	nearest	9.747	3.253	0.998
	R-track	nearest	28.827	4.893	1.000
	R-track	restart	41.376	14.594	0.999
SARSA	L-track	nearest	13.447	3.479	0.967
	O-track	nearest	31.628	3.190	0.986
	W-track	nearest	9.928	3.557	0.998
	R-track	nearest	28.162	2.693	1.000
	R-track	restart	48.676	23.797	0.998
Value Iteration	L-track	nearest	11.570	1.352	1.000
	O-track	nearest	24.810	2.753	1.000
	W-track	nearest	9.010	0.911	1.000
	R-track	nearest	25.910	2.582	1.000
	R-track	restart	38.510	15.501	1.000

Table 1: Mean steps-to-finish, standard deviation, and success rate across racetracks, crash modes, and algorithms. Each value is computed over 100 greedy evaluation trials per trained model, excluding episodes that reached the step limit.

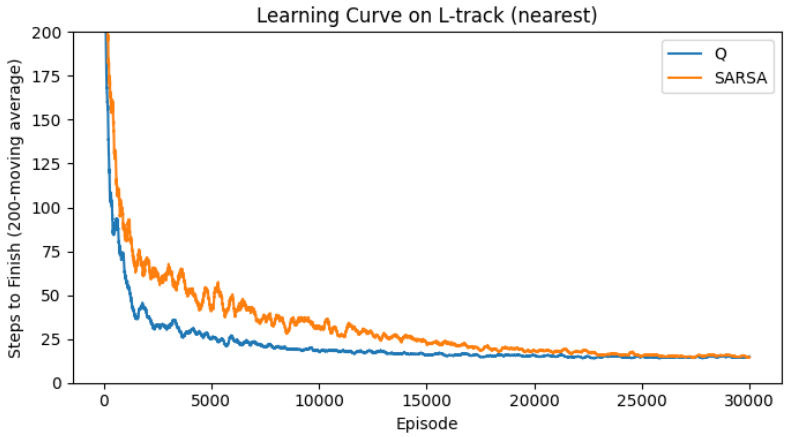

Figure 5: Learning Curve for Q-Learning and SARSA Agents on the L-track under the Nearest crash mode

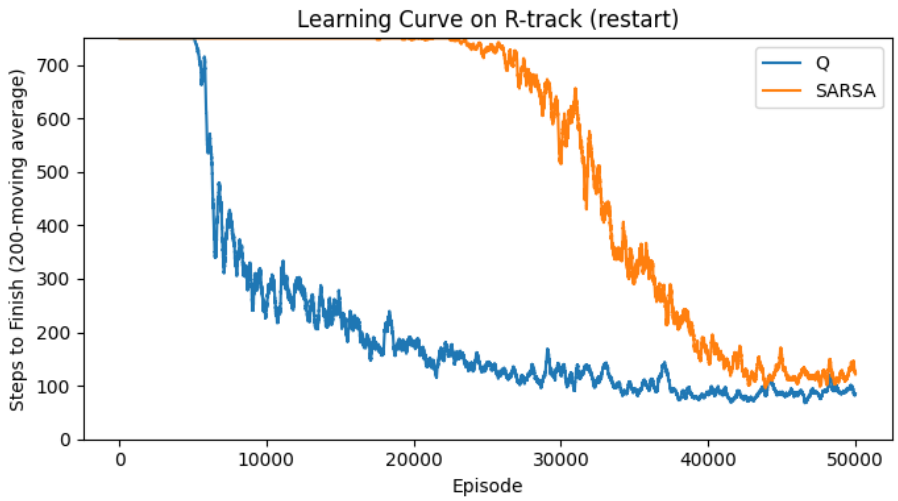

Figure 6: Learning Curve for Q-Learning and SARSA Agents on the R-track under the Restart crash mode

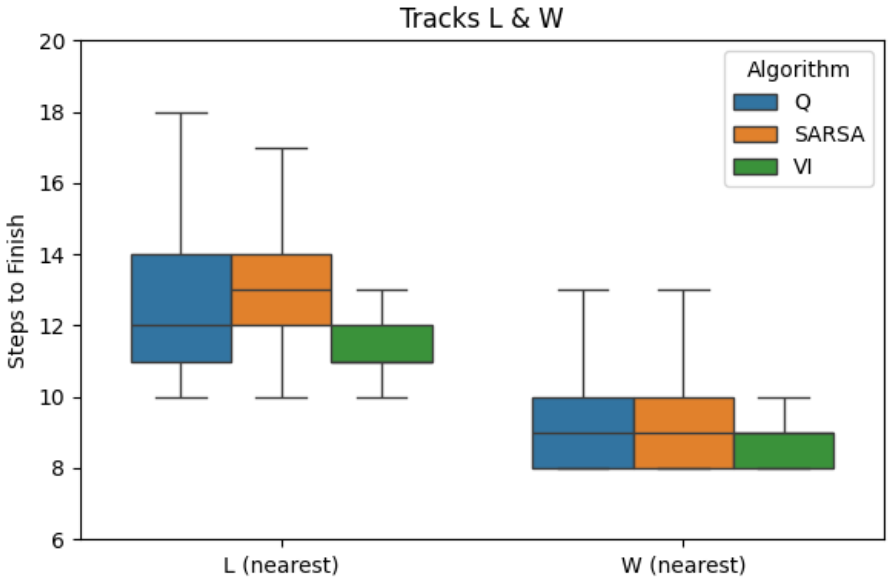

Figure 7: Steps-to-finish distributions for L-track and W-track under different algorithms and crash modes.

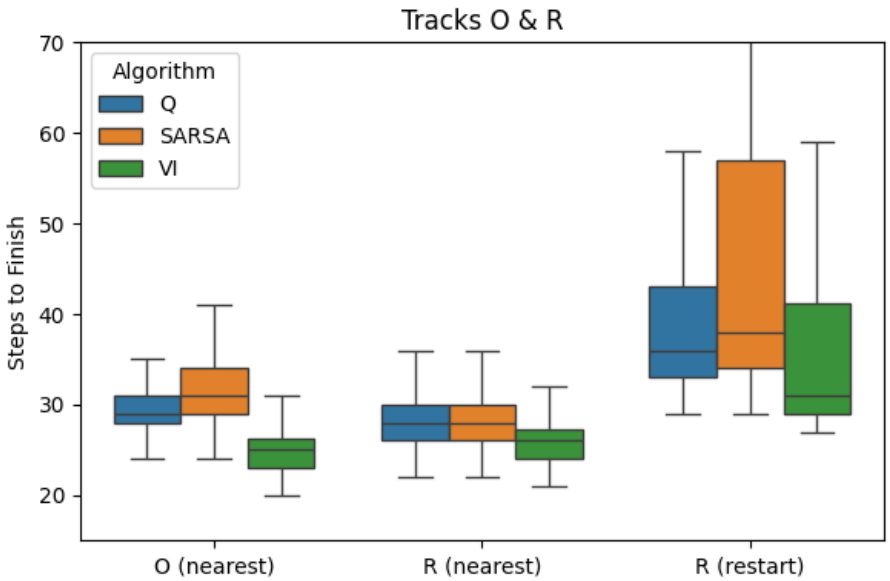

Figure 8: Steps-to-finish distributions for O-track and R-track under different algorithms and crash modes.

5 Discussion

The experimental results are summarized above. Table 1 reports the mean and standard deviation of steps-to-finish over 100 greedy trials for each algorithm-track-mode combination, along with the empirical success rate. Figures 5 and 6 show the full distribution of trial outcomes, and Figures 7 and 8 show learning curves for the L-track (nearest mode) and R-track (restart mode). The rest of the learning curves where omitted for brevity, since the pattern is the same.

5.1 Hypothesis 1

Across all tracks, Value Iteration (VI) was hypothesized to produce the lowest average steps-to-finish. The results strongly support this hypothesis. VI achieved the best mean performance on every track-mode pair, often by substantial margins (e.g., O-track nearest: VI = 24.8 vs. Q-learning = 29.6, SARSA = 31.6). The advantage was most pronounced in tracks with more constrained geometry, such as O-track and R-track, where the model-based nature of VI allowed it to propagate accurate value estimates across the state space without the sampling noise inherent in the other methods. The boxplots further confirm this consistency: VI's interquartile ranges were minimal (with the exception of R-track restart), reflecting nearly identical results across trials. This stability is a direct result of VI's deterministic updates given the fixed environment model.

5.2 Hypothesis 2

It was hypothesized that SARSA would yield lower variance in steps-to-finish than Q-learning, while Q-learning would achieve a lower mean after sufficient training. The experimental evidence does not support either claim. SARSA generally had both a higher mean cost and a comparable or higher variance. For example, on L-track nearest, SARSA averaged 13.4 steps (SD = 3.48) compared to Q-learning's 12.8 steps (SD = 2.4), and on W-track nearest SARSA averaged 9.9 steps (SD = 3.6) versus Q-learning's 9.7 (SD = 3.3). The boxplots show that Q-learning's variability was generally lower than SARSA's. The learning curves reveal an additional difference: in all tested cases, Q-learning converged to near-optimal performance in fewer episodes than SARSA, with both algorithms ultimately achieving similar asymptotic performance levels. This suggests that Q-learning's off-policy updates may not have introduced the instability often expected in hazardous environments, at least under the ϵ-greedy exploration schedule used here.

6 Conclusion

This project compared the performance of three reinforcement learning algorithms (Q-learning, SARSA, and Value Iteration) on multiple racetrack navigation tasks under two crash-handling modes. Using tuned hyperparameters for each setting, performance was evaluated in terms of steps-to-finish, variance across trials, and convergence speed. The results confirmed that Value Iteration consistently produced the lowest average steps-to-finish and the most stable performance, reflecting the advantages of model-based planning in deterministic, fully known environments. In contrast, SARSA did not demonstrate the hypothesized advantage in variance and generally underperformed Q-learning in both efficiency and convergence rate. The learning curves further highlighted Q-learning's faster improvement during training compared to SARSA, despite their similar final performance levels. Future work could explore alternative exploration schedules, stochastic track variants, or partially observable settings to test whether these trends persist under more challenging conditions.